

DBMS Unit 3

Q1. Explain the concept of a Transaction in DBMS. Discuss the ACID properties in detail with suitable examples. (15 Marks)

Transaction: A transaction is a single logical unit of work that accesses and possibly modifies the contents of a database. It consists of a sequence of operations (typically Read and Write) that must be executed completely or not at all to maintain database consistency.

To maintain consistency before and after the transaction, the database enforces four crucial rules known as the **ACID properties**:

- **Atomicity (The 'All or Nothing' Rule):** This ensures that either the entire transaction takes place at once or it doesn't happen at all. Transactions do not occur partially. If a failure occurs mid-execution, the DBMS **aborts** the transaction and rolls back any changes. Only if all steps succeed does the DBMS **commit** the transaction.

Example: If ₹1000 is deducted from Account X to be added to Account Y, and the system crashes before the addition, Atomicity ensures the ₹1000 deduction is fully reversed.

- **Consistency:** This ensures that the database moves from one valid state to another. Integrity constraints must be maintained.

Example: The total sum of balances in Account X and Y before the transaction must exactly equal the total sum after the transaction.

- **Isolation:** Multiple transactions can execute concurrently without leading to inconsistency. The changes made by one transaction are not visible to others until committed.

Example: If two operations run concurrently on different accounts, they execute independently without interference, yielding results identical to serial execution.

- **Durability:** Once a transaction completes successfully (commits), the changes made to the database persist permanently, even in the event of system failures, software crashes, or power outages. Updates are securely written to non-volatile storage.

Q2. What is Concurrency Control? Explain the various problems that occur with concurrent execution of transactions with examples. (15 Marks)

Concurrency Control is the management procedure required to control the simultaneous execution of operations in a database without affecting data consistency. While interleaved execution improves CPU utilization and overall throughput, uncontrolled interleaving leads to severe data anomalies.

The major problems associated with uncontrolled concurrent execution include:

- **1. Dirty Read Problem (Write-Read Conflict):** Occurs when one transaction reads data that has been updated by another uncommitted transaction. If the updating transaction fails and rolls back, the first transaction has read invalid or "dirty" data.
Example: T1 updates Account A from ₹300 to ₹350 but hasn't committed. T2 reads the ₹350. T1 crashes and rolls back to ₹300. T2 is now working with a non-existent value.
- **2. Incorrect Summary Problem:** Occurs when a transaction calculates an aggregate summary function (like SUM) on records while another transaction is modifying them. The aggregate calculation includes some values before their update and others after.
Example: T1 transfers ₹50 from A to B. T2 is summing total balances. T2 reads A after the deduction but reads B before the addition, resulting in an incorrect summary (short by ₹50).
- **3. Lost Update Problem (Write-Write Conflict):** Occurs when two transactions perform read and write operations on the same data item, and one transaction's write completely overwrites and destroys the other's update.
Example: T1 and T2 both read balance A as ₹300. T1 deducts ₹50 (writes ₹250). T2 adds ₹100 and writes ₹400, completely overwriting and losing T1's deduction.
- **4. Unrepeatable Read Problem (Read-Write Conflict):** Occurs when a transaction reads the same data item twice during its execution but gets different values because another transaction successfully modified the data in between the reads.

Q3. What are Schedules in DBMS? Differentiate between Conflict Serializability and View Serializability. (10 Marks)

Schedule: A schedule is the chronological execution sequence of operations from one or more transactions. It preserves the order of operations within individual transactions while safely interleaving operations from different transactions.

To ensure database consistency during concurrent execution, schedules are tested for **Serializability**—ensuring the interleaved execution yields the exact same result as a pure serial execution.

Feature	Conflict Serializability	View Serializability
Definition	A schedule is conflict serializable if it can be transformed into a serial schedule by swapping its non-conflicting operations.	A schedule is view serializable if it is "view equivalent" to a serial schedule (matching initial reads, update reads, and final writes).

Strictness Level	Highly strict. Every conflict serializable schedule is automatically a view serializable schedule.	Less strict. A schedule can be view serializable without being conflict serializable (especially if it contains blind writes).
Blind Writes	Cannot definitively prove serializability if cycles are formed due to blind writes.	Specifically designed to handle and validate schedules that contain blind writes (writing without reading).
Validation Method	Tested by drawing a Precedence Graph. If there are no cycles, it is conflict serializable.	Tested by comparing dependency graphs and checking Initial Reads, Final Writes, and intermediate sequence updates.

Q4. Discuss the different types of File Organization in DBMS. Differentiate between Heap File Organization and Sorted File Organization. (12 Marks)

File Organization refers to the logical arrangement of data in a storage system. It determines how records are stored on a disk and retrieved efficiently. Based on the Cost Model (evaluating Disk I/O, pages, and processing time), the Database Administrator selects the optimal method.

Primary Types of File Organization:

1. **Heap (Pile) File Organization:** Records are placed randomly in the file wherever there is space.
2. **Sorted (Sequential) File Organization:** Records are strictly ordered sequentially based on the value of a search key.
3. **Hashed File Organization:** A hash function maps a search key to a specific physical bucket or address.
4. **Clustered File Organization:** Groups related records from multiple relations into the same block to minimize Disk I/O.

Aspect	Heap File Organization	Sorted (Sequential) File Organization
Data Arrangement	Unordered. Data is stored sequentially as it arrives at the end of the file.	Strictly Ordered. Data is stored in a sorted order based on a designated key field.
Insertion Efficiency	Highly efficient. New records are simply appended to the last block.	Costly and inefficient. Requires finding the correct position and shifting subsequent records to maintain order.
Search/Query Speed	Slow. Requires a full linear scan of all blocks unless searching by a specific ID.	Extremely Fast for key searches. Utilizes Binary Search to locate records and efficiently processes range queries.

Ideal Use Case	Best for small datasets or logs where bulk insertions are vastly more frequent than complex read queries.	Best suited for applications needing sequential access, range queries, and heavy reporting tasks.
-----------------------	---	---

Q5. Explain B+ Tree Index Files. What are the advantages of B+ Trees over indexed-sequential files, and explain the structure of Leaf and Non-Leaf nodes. (12 Marks)

A **B+ Tree** is an advanced, multi-level balanced index structure used extensively in DBMS to store, search, and retrieve data efficiently from disk. It acts as a robust alternative to indexed-sequential files by automatically reorganizing itself with small, local changes in the face of insertions and deletions.

Advantages over Indexed-Sequential Files:

- **No Performance Degradation:** Unlike indexed-sequential files, B+ tree performance does not degrade as the file grows because overflow blocks are not endlessly chained.
- **No Full Reorganization Needed:** Periodic and complete reorganization of the entire file is absolutely not required to maintain high performance.
- **Logarithmic Search Efficiency:** Searches are highly efficient. For K search keys and node size n , the tree height is tightly bounded, ensuring consistent lookup times.

Structure of B+ Tree Nodes:

- **Leaf Nodes:** Leaf nodes are located at the bottom-most level of the tree and are completely balanced. They contain the actual data pointers. Crucially, leaf nodes are linked sequentially (like a linked list), which enables extremely fast sequential and range queries without traversing back up the tree.
- **Non-Leaf (Internal) Nodes:** These act purely as a multi-level sparse index guiding the search down the tree. They store only search keys and pointers to child nodes—never the actual data records.

Q6. Explain the B-Tree / B+ Tree Algorithm (Insertion and Search) with its structural properties. (15 Marks)

A **B+ Tree** is an advanced, self-balancing, multi-level tree structure used heavily in DBMS to efficiently store and retrieve records. It ensures that the tree remains balanced even after numerous insertions and deletions, guaranteeing logarithmic $O(\log N)$ time complexity for searching.

Structural Properties (Order 'm'):

- **Internal Nodes:** Can contain up to $m-1$ search keys and m children pointers. They do not store actual data; they only guide the search traversal down the tree.
- **Leaf Nodes:** Contain the actual data records (or pointers to physical records). All leaf nodes are located at the exact same level (hence "balanced") and are sequentially linked like a Linked List to allow extremely fast range queries.

The Search Algorithm:

1. Start at the Root Node. Compare the search key (K) with the keys inside the node.
2. If $K < \text{Key}_1$, follow the left-most pointer. If $\text{Key}_1 \leq K < \text{Key}_2$, follow the middle pointer, and so on.
3. Repeat this traversal downwards until a Leaf Node is reached.
4. Search linearly within the Leaf Node for the exact key. If found, retrieve the record; otherwise, the record does not exist.

The Insertion Algorithm:

1. Traverse the tree to find the correct Leaf Node where the new key should be placed.
2. Insert the key in sorted order. If the node has fewer than m keys, insertion completes.
3. **Node Split:** If the node overflows (contains exactly m keys), split it into two nodes. Keep the first half in the original node and move the second half to a new right-sibling node.
4. Copy the smallest key of the new right node and push it UP to the parent node. If the parent also overflows, recursively split up the tree until the root splits (which increases the tree's height by 1).

Q7. Discuss Serializability in DBMS. Explain Conflict operations (Conflict Read/Write). (12 Marks)

Serializability is a concurrency control concept ensuring that concurrent execution of multiple transactions results in a database state that is identical to some purely serial execution of those same transactions. It guarantees correctness while allowing maximum interleaving.

To determine if a schedule is serializable, we analyze **Conflicting Operations**. Two operations conflict if they belong to different transactions, access the exact same data item, and at least one of them is a Write operation.

Types of Conflicts:

- **Read-Write (RW) Conflict - Unrepeatable Read:** T1 reads data 'A'. Before T1 commits, T2 writes (modifies) 'A'. If T1 tries to read 'A' again, it gets a completely different value.

- **Write-Read (WR) Conflict - Dirty Read:** T1 writes 'A'. T2 reads 'A'. T1 then aborts and rolls back. T2 has now read invalid, "dirty" data that officially never existed in the database.
- **Write-Write (WW) Conflict - Lost Update:** T1 writes 'A'. Before T1 commits, T2 also writes 'A'. T1's update is completely overwritten and lost without ever being read.

Example & Precedence Graph: To check for Conflict Serializability, the DBMS constructs a Precedence Graph mapping the RW, WR, and WW conflicts as directed edges between transactions. If the graph contains a cycle (e.g., $T1 \rightarrow T2 \rightarrow T1$), the schedule is NOT conflict serializable. If it is acyclic, the schedule is strictly conflict serializable.

Q8. Describe B+ Tree as a Dynamic Index Structure in DBMS. (12 Marks)

A **Dynamic Index Structure** is an **indexing mechanism** that automatically adapts **its physical layout in response to data modifications** (insertions and deletions) **without requiring complete file reorganization**. The **B+ Tree** is the **most prominent dynamic index used in modern DBMS**.

Why is it Dynamic?

- In older **static indexing methods** (like ISAM - Indexed Sequential Access Method), as data grew, **overflow blocks were chained endlessly**, **destroying performance**. Eventually, the database had to be taken **offline to rebuild the index**.
- A B+ Tree solves this by dynamically **growing and shrinking from the root**. When a block overflows, it simply splits into two, **maintaining its perfectly balanced $O(\log N)$ structure automatically**.

Key Advantages as a Dynamic Index:

- **No Reorganization Required:** The tree maintains its structural integrity continuously, saving immense DBA maintenance time.
- **Optimized Disk I/O:** Because the tree is "fat and short" (high fan-out / large 'm' value), the depth of the tree rarely exceeds 3 or 4 levels, even for millions of records. Searching only requires 3-4 disk accesses.
- **Excellent for Range Queries:** Because leaf nodes are horizontally linked, a query like ``SELECT * FROM Employees WHERE Salary BETWEEN 40000 AND 60000`` is incredibly fast. The DBMS traverses down to 40,000, then simply walks the linked list horizontally until it hits 60,000.

Q9. What is Lock-Based Concurrency Control? (10 Marks)

Lock-Based Concurrency Control is a protocol employed by the Database Manager to maintain isolation and serializability. Before a transaction can read or write a data item, it must request and acquire a specific lock from the Lock Manager.

Types of Locks:

- **Shared Lock (S-Lock):** Acquired when a transaction only wants to READ a data item. Multiple transactions can hold a Shared lock on the same item simultaneously because reading does not compromise consistency.
- **Exclusive Lock (X-Lock):** Acquired when a transaction wants to WRITE (modify) a data item. Only one transaction can hold an Exclusive lock on an item. If granted, no other transaction can read or write to it until the lock is released.

Lock Compatibility:

- If T1 holds a Shared lock, T2 can also be granted a Shared lock.
- If T1 holds a Shared lock, T2's request for an Exclusive lock is DENIED (must wait).
- If T1 holds an Exclusive lock, T2's request for ANY lock (Shared or Exclusive) is DENIED (must wait).

Challenges: While locking ensures consistency, it introduces the risk of **Deadlock** (where T1 waits for T2's lock, and T2 waits for T1's lock, causing a permanent freeze) and **Starvation** (where a transaction waits indefinitely because other transactions keep jumping the queue).

Q10. Explain the Two-Phase Locking (2PL) Protocol and its variations. (15 Marks)

The **Two-Phase Locking (2PL) Protocol** is a strict concurrency mechanism that dictates how a transaction acquires and releases locks. If all transactions strictly follow 2PL, the DBMS mathematically guarantees conflict serializability.

The Two Phases:

1. **Growing Phase:** A transaction can acquire new locks (Shared or Exclusive) on data items but cannot release any lock. The moment a transaction releases its very first lock, the growing phase permanently ends.
2. **Shrinking Phase:** A transaction can release its existing locks but is strictly forbidden from acquiring any new locks.

Problems with Basic 2PL: Basic 2PL does not prevent Deadlocks, nor does it prevent **Cascading Rollbacks** (where a transaction aborts, forcing all transactions that read its uncommitted data to also abort).

Advanced Variations of 2PL:

- **Strict Two-Phase Locking (Strict 2PL):** To completely eliminate cascading rollbacks, Strict 2PL mandates that a transaction must hold onto all of its **Exclusive (X) locks** until the very end of the transaction (until it either commits or aborts). This prevents any other transaction from reading its uncommitted changes (Dirty Reads).

- **Rigorous Two-Phase Locking (Rigorous 2PL):** This is an even stricter protocol where a transaction must hold **ALL locks (both Shared and Exclusive)** until it commits or aborts. This makes implementation extremely simple and prevents unrepeatable reads, but it significantly reduces the overall concurrency level of the system.